TEMA 6 – Utilización del DOM

Objetivo del tema

En este tema aprenderemos sobre los Eventos y los Formularios

6.1 Utilización del modelo de eventos del documento (DOM)

Consulta la presentación **UD06 - Utilización del DOM**.

El **Modelo de Objetos del Documento (DOM)** es el estándar de la W3C que define cómo acceder a los documentos HTML, XML, etc. Sirve como interfaz de programación a aplicaciones (API) del W3C. De forma simplificada, podemos decir que DOM permite tratar elementos HTML desde JavaScript como si fuesen Objetos. De esta forma, un script puede acceder y actualizar el contenido de una página web dinámicamente.

Originalmente, la W3C organizó la arquitectura DOM en **tres niveles de especificación y especialización**.

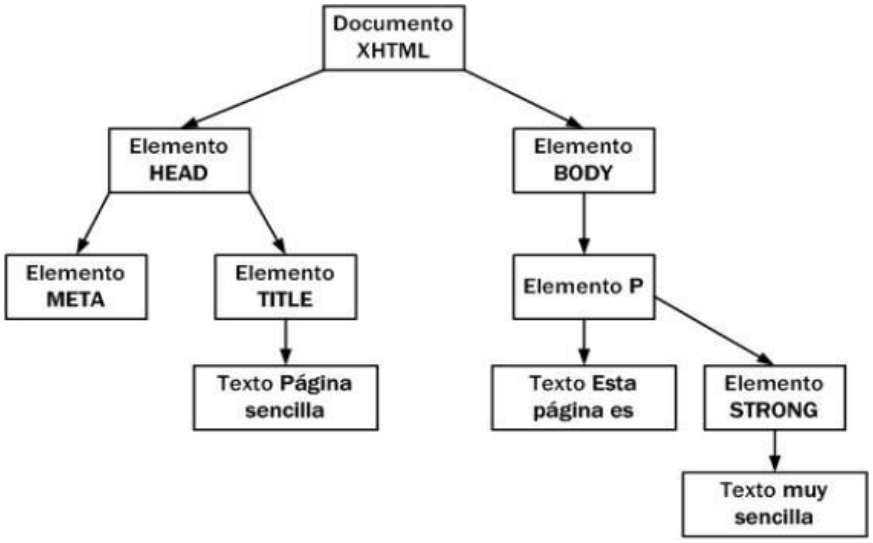
- DOM Core – Para cualquier documento estructurado.
- XML DOM – Estándar para documentos XML. Versión especializada del DOM Core.
- HTML DOM – Estándar para documentos HTML. Versión especializada del DOM Core.

En pocas palabras, DOM trata un documento HTML como si fuese un árbol invertido. Partiendo de un único nodo raíz, todos los demás elementos 'cuelgan' de ese nodo raíz en orden. No hay límite en cuanto al número de hijos que puede tener un nodo. Finalmente, un nodo sin hijos se denomina hoja.

Tomemos como ejemplo este HTML sencillo:

```
<!DOCTYPE html PUBLIC "//W3C//DTD XHTML 1.0 Transitional//EN"
    "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
<html xmlns="http://www.w3.org/1999/xhtml">
    <head>
        <meta http-equiv="Content-Type"
        content="text/html; charset=iso-8859-1" />
        <title>Página sencilla</title>
    </head>
    <body>
        <p>Esta página es <strong>muy sencilla</strong></p>
    </body>
</html>
```

La forma que tiene DOMO de 'ver' y tratar este documento es la siguiente:



La forma que tenemos de recorrer esta estructura desde JavaScript es simple. Disponemos de una serie de funciones que nos permite recorrer el árbol, recuperando nodos, sus hijos, y accediendo a sus propiedades.

De forma simplificada, para cambiar el título de esta página web podríamos hacer:

1- Accedemos a la raíz, que es Documento XHTML.
2- Obtenemos sus nodos hijos, que son Elemento Head y Elemento Body.
3- Recuperamos Elemento Head.
4- Recuperamos sus nodos hijos, que son Elemento Meta y Elemento Title.
5- Recuperamos Elemento Title.
6- Cambiamos el texto de Elemento Title.

Mediante esta forma de trabajar, podemos hacer cosas como **modificar** nodos, **añadir** nodos nuevos, o incluso **eliminarlos**. Nótese que estas modificaciones NO se hacen sobre el propio documento HTML, sino sobre la versión en memoria del navegador. Una recarga de la página desharía todos estos cambios.

Para conservar los cambios entre recargas sin depender de un servidor backend, se deben guardar los cambios usando tecnologías de almacenamiento web como **localStorage / sessionStorage**.

JavaScript crea un objeto llamado **Node** para para manipular cada uno de los nodos de la estructura. En la presentación **UD06 - Utilización del DOM** tienes, en el apartado 6.2, un listado y descripción exhaustiva de todos los métodos y propiedades que nos ofrece DOM **para recorrer** el árbol de nodos.

Para el caso de ejemplo que hemos visto:

```
Nodo raíz:
var obj_html= document.documentElement;

Primer y útlimo hijo del nodo:
var obj_head= obj_html.firstChild;
var obj_body= obj_html.lastChild;

Acceso a nodos desde el índice de un array:
var obj_head= obj_html.childNodes[0];
var obj_body= obj_html.childNodes[1];

Acceso al número de hijos (longitud del array):
var numeroHijos= obj_html.childNodes.length;
```

En la presentación **UD06 - Utilización del DOM** tienes, en el apartado 6.3, un listado y descripción exhaustiva de todos los métodos y propiedades que nos ofrece DOM **para acceder** directamente a un nodo y a sus atributos.

6.2 Manipulando elementos del árbol

En la presentación **UD06 - Utilización del DOM** tienes, en el apartado 6.3, un listado y descripción exhaustiva de todos los métodos y propiedades que nos ofrece DOM **para añadir** nodos a un árbol.

Para añadir un nuevo nodo, que puede ser virtualmente cualquier elemento HTML, tenemos que 'colgarlo' de otro nodo existente. Obviamente, en ese caso tendremos que recuperar ese nodo en primer lugar.

Un ejemplo simple. Añadamos un <h1> a <body> con un atributo class:

```
// Creamos el elemento h1
var h = document.createElement("h1") ;
// Crea el nodo de texto
var t = document.createTextNode("Hola, mundo");
//añade el texto al elemento h1
h.appendChild(t);

// Ahora, tenemos en memoria <h1>Hola, mundo</h1>

// Creamos el atributo class para h1 y le asignamos el valor
var att = document.createAttribute("class");
att.value = "prueba";
//añade el atributo al nodo h1 creado
h.setAttributeNode(att);

// Ahora, tenemos en memoria <h1 class="prueba">Hola, mundo</h1>

// Recuperamos elemento body y añadimos hi al final
var b = document.body;
b.appendChild(h);
```

Ejercicio DOM - I

Dispones de un fichero llamado UD06_DOM1_Apellido_Nombre.html para realizar este ejercicio.

Introduce en el apartado de *script* el código necesario para extraer y mostrar por pantalla:

1. El número de párrafos de la página.
2. El texto del segundo párrafo.
3. El número de enlaces de la página.
4. La dirección del primer enlace.
5. La dirección del penúltimo enlace.
6. El número de enlaces que apuntan a /wiki/Balaenidae
7. El número de enlaces del primer párrafo.

Para agregar texto en la página de forma dinámica (DOM) utiliza una etiqueta div con el id=info y añade en ella toda la información detallada mediante:

```
let info = document.getElementById("info");
info.innerHTML = "Texto informativo";
```



SO SAYS DOOM!

Ejercicio DOM - II

Realiza los siguientes pasos:

1. Crea una página web que tenga un listado de tipo <ul> con un <li> de muestra.

2. Introduce un botón en la página que, cuando lo pulses, te muestre un prompt para que el usuario introduzca un texto.

3. Una vez cerrado el prompt el valor se añadirá como un nuevo <li> a la lista creada.

4. Añade dos botones más con texto "Borrar primer li" y "Borrar último li" de modo que cuando pulses el primer botón borre el primer elemento de la lista y cuando pulses el último borre el último elemento de la lista.

Ejercicio DOM - III

Crea de manera dinámica (al cargarse la página) el formulario que definimos en el ejercicio de la unidad 5. Para ello deberás crear los nodos elementos, añadirlos como hijos de sus padres, crear los hijos de éstos y añadirlos también y así sucesivamente.

Ten en cuenta que el formulario deberá tener los atributos necesarios para que tenga la misma funcionalidad que el que creaste en *html*.

No olvides añadir las etiquetas *<label>* a cada uno de los elementos.

Ejercicio DOM - IV

Crea una aplicación que te permitirá generar tus propios formularios de manera dinámica. Para ello dibuja una fila con varios botones, que realicen lo siguiente:

- Crear un *input* de tipo *texto*. Le preguntará al usuario mediante un *prompt* qué nombre (atributo *name*) tiene el *input*.
- Crear un *input* de tipo *password*. Le preguntará al usuario mediante un *prompt* qué nombre (atributo *name*) tiene el *input*.
- Crear un *textarea*. Le preguntará al usuario mediante un *prompt* el nombre y generará automáticamente un *textarea* de 40 columnas y 5 filas.
- Crear un *label*. Preguntará al usuario a qué *input* va referido (atributo *for*). Además se preguntará qué texto mostrará dicho *label*.
- Crear una imagen. Preguntará al usuario qué ruta tiene la imagen (atributo *src*).
- Crear un *checkbox*. Preguntará al usuario el nombre y el valor (atributos *name* y *value*).
- Crear un *radio*. Preguntará al usuario el nombre y el valor (atributos *name* y *value*).
- Crear un botón (*submit*). Preguntará al usuario el nombre y el valor (atributos *name* y *value*).